OPEN AGENT HACKATHON 2026

GreenOps Verify

Complete Hackathon Blueprint

Cloud optimization.
AI efficiency.
Evidence before action.

A GreenOps-inspired implementation plan with required Zetaris and Meterless integration, a focused AI Efficiency Lab, and an approval-to-verification workflow.

KEEP	ADD
Audit cloud resources	Federated, cited evidence through Zetaris
Forecast workload and compare impact	Independent trials and critic-driven revisions
Optimize and report	Persistent experiment memory through Meterless
Estimate environmental impact	Scoped approval, fresh-state checks and verification

PREPARED 16 OCTOBER 2026 | VERSION 1.0

Design proposal + static repository review. No deployment, sponsor access, savings or production-readiness claim is made by this document.

Original GreenOps inspiration: Aishwarya Nathani and Nikhil Mankani. This proposed project is not their award-winning submission and is not presented as affiliated with them. [P1, R1]

01 / EXECUTIVE DECISION

The decision in one page

Keep the GreenOps concept. Make the new contribution evidence-backed optimization, not a technology-logo swap.

Working title: GreenOps Verify. The title is a planning label, not a verified available brand and not a claim of affiliation with the original authors. The product helps an engineering or cloud-operations lead reduce avoidable cloud and AI costs, estimate supported carbon impacts, and approve changes only after workload checks.

Decision	Recommended choice
Core product	A cloud optimization desk: audit, forecast, compare, trial, approve, verify and report.
SpendProof addition	One AI Efficiency Lab for a support workflow. Test redundant-call removal and context selection before considering a model switch.
Primary track	Track 02 for an independently implemented entry. Ask organizers about Wildcard eligibility before extending the existing third-party project.
Core stack	Python + Google ADK + FastAPI + Streamlit; Zetaris evidence access; Meterless H-MEM TypeScript sidecar; NVIDIA-hosted model.
Build boundary	GCP only; six sample VMs; one disposable/sandbox action; one AI workflow; a maximum of three candidates per path (six total).
Success demonstration	The system rejects an attractive but invalid saving, revises it, verifies an approved change, and remembers the outcome after restart.

What this package is—and is not

This is a researched design and static code review, prepared before the build window. It does not contain a deployed application, tested sponsor connection, achieved savings, or an eligibility ruling. All architecture, thresholds, examples, timelines and work items are proposed unless explicitly identified as an observed source fact.

Do not pitch two unrelated products. Pitch one optimization system with two bounded levers: cloud resources and AI-workflow efficiency.

02 / READING MAP

How to use this blueprint

Section	What the team should decide
03–04 | Product and demo	Who uses it, the one decision it supports, and the main end-to-end example.
05–06 | Baseline and eligibility	What the repository actually contains; permission, attribution and track selection.
07–09 | Scope and architecture	The committed minimum, technology changes and interacting agent roles.
10–11 | Sponsor integration	Where Zetaris and Meterless are essential, how to prove that, and which APIs are our own.
12–15 | Optimization logic	Forecasting, workload constraints, cost/carbon accounting and AI quality experiments.
16–18 | Execution and data	Approvals, state checks, schema contracts and reproducible sample scenarios.
19–21 | Delivery	Tests, team allocation, deployment, runtime and spend controls.
22–24 | Submission and risks	Four-minute demo, deck outline, attribution, claims and fallback policy.
25–27 | Implementation and sources	File-level changes, first work items, technical references and source limitations.

Evidence labels used throughout

[H] references are the user-supplied event guides. [R] references point to selected original repository files at a recorded commit. Other labels point to primary product documentation or the original Devpost description. Full references appear at the end; hyperlinks are clickable in Word and PDF.

Observed means visible in the inspected source. Proposed means the recommended build. Synthetic means invented scenario data. Estimated means a calculation using declared assumptions. Measured means an actual observation from a run with a timestamp and version. Do not replace one label with another in the final demo.

Practical starting order

The product owner should read sections 03, 06, 07 and 22 first. The data/integration builder should start with 10, 11 and 17. The agent builder should start with 09 and 12–16. The demo owner should own section 19's acceptance checklist from the first day, not the last.

Use the companion Team Brief for alignment and the Developer Handoff Markdown for implementation contracts and Cursor instructions.

03 / PRODUCT

The complete product idea

Target user: the engineering lead at a small AI-enabled software company. Their support service runs on GCP and calls an external model. The team needs a practical answer to: “Which changes should we make this week to remove waste without violating our service requirements?”

Keep the original GreenOps foundation

The original submission describes infrastructure recommendations, utilization forecasts, cost/carbon comparisons, a VM change workflow and executive reports. Its stated implementation uses Google ADK, Gemini/Vertex AI, BigQuery, Climatiq, Streamlit and Google reporting tools. These are original project descriptions, not verified production benchmarks. [P1, R1]

Add a more defensible decision layer

Our proposed version makes every recommendation inspectable. It records the source observations, proposed change, relevant workload constraints, experiment results, approval and subsequent verification. A rejected plan is retained with the reason it failed. A later run can reuse that context, but must recheck the current facts.

The AI Efficiency Lab shares the same decision record and approval flow. It does not become a separate subscription manager. It asks whether the support workflow can complete the same required work with fewer redundant calls or less irrelevant context. Quality is checked independently of the model proposing the change.

What the user receives

A ranked review queue, a side-by-side comparison of candidates, evidence links, a workload-test result, a cost estimate with assumptions, a separately labelled carbon estimate, and a scoped approve/reject control. After execution, the same page shows the resulting state and any unresolved verification steps.

Product promise: “Find avoidable cloud and AI waste. Test the change. Show the evidence. Apply only the approved scope.” Do not promise zero downtime, guaranteed savings, certified carbon reductions or production safety.

04 / WORKED SCENARIO

One story for the whole demo

A synthetic GCP support service has an apparently idle worker, an over-provisioned staging VM and an inefficient AI support workflow.

Step	What happens on screen
1 | Request	“Review this service’s cloud and AI spend. Propose savings while preserving its workload, latency and support-policy requirements.”
2 | Investigate	Zetaris combines cloud history, AI outcomes and policy/price records. The UI shows source IDs and observation times.
3 | Challenge cloud plan	An 8-vCPU worker averages only 10% CPU, but has a recorded peak demand of 6.4 vCPU. A proposed 2-vCPU target is rejected.
4 | Revise	The planner leaves that worker unchanged and trials a separate staging candidate with adequate CPU/memory headroom.
5 | Challenge AI plan	Aggressive context removal lowers call usage but drops a premium-return rule. The independent test fails it.
6 | Select bounded change	A revised policy-aware retrieval configuration is evaluated. Only a candidate that passes the locked checks goes for approval.
7 | Execute and verify	The operator approves a versioned change in the sandbox. The executor rechecks state, applies only that change, and reads the result back.
8 | Continue after restart	Meterless recalls the rejected plan and approved result. A changed policy version makes the old validation stale and triggers a new test.

The vCPU figures above are a deliberately constructed test case, not live measurements or a vendor performance claim. The system must derive each result from the supplied records; do not hardcode the narrative as agent output.

The visual takeaway

Put three candidate cards side by side: rejected for workload risk, rejected for answer-policy failure, and eligible for a bounded trial. The judge should see why the cheapest-looking option is not necessarily the correct choice.

Cloud optimization stays the main story. The AI experiment demonstrates that the same evidence-and-verification discipline also applies to AI spending.

05 / OBSERVED BASELINE

What the repository review found

Inspected baseline: niksm7/GreenOps, master, commit db38811fb653631892e6cbbe1275793bd9906858. Reviewed the tree, README and selected coordinator, advisor, forecast, impact and executor modules. This is a static review, not an exhaustive security audit or a runtime test. [R1–R6]

Observed item	Why it matters / proposed response
Advisor is a SequentialAgent	Retain useful specialist roles, but add a critic-to-planner revision path. The guide explicitly values feedback and recovery. [R3, H1 §3]
Forecast gate divides each sum by 7	Empty lists produce zero averages; missing values are not checked in that function. Validate coverage and units; missing evidence must block eligibility. [R4]
Gate ignores proposed capacity	A low average on a large VM does not establish adequate headroom on a smaller VM. Evaluate absolute demand against the actual target and test the workload. [R4]
Prompt-level confirmation exists	The root asks whether to execute. However, the low-level VM-change tool does not itself enforce an approval token or the preceding safety result. Put those checks in application code. [R2, R4]
Carbon duration mismatch	The impact helper defaults to a 24-hour request, while the agent instruction multiplies its returned total by 24 again. This creates an overcounting risk; it is not a verified claim about every live answer. [R5]
Carbon result key is machine type	Same-type comparisons across two regions can overwrite a result. Use explicit scenario IDs plus provider, region, type and duration. [R5]
Pricing is HTML-scraped	Replace the fragile page-selector dependency with a bounded, dated official price snapshot or documented account rates. [R5, G2]
Project/model paths are fixed	Remove original project assumptions; lazy-load clients; supply own credentials and verify current model/dependency availability. [R2, R4, R6]

The inspected tree includes an .env file and compiled/cache artefacts. This is a repository-hygiene observation, not proof that a live secret is exposed. Do not reuse original credentials, cloud resources or compiled files. No cloud actions or credentials were tested during this review.

06 / ELIGIBILITY GATES

Reuse, track and deadline decisions

Two valid planning paths—not an automatic permission to copy

No licence grant was found in the inspected repository tree or README. GitHub explains that a public repository is not automatically licensed for modification and redistribution. Obtain permission covering the relevant code/assets, or independently implement the concept without copying its code, visual assets or submission wording. Attribution alone is not a substitute for permission. [R1, L1]

Path	Track and evidence
A | Independent implementation — default	Keep conceptual inspiration; create original code during the build window. Target Track 02 and declare the original GreenOps reference plus any permitted pre-existing components.
B | Authorized derivative	First obtain reuse permission and an organizer ruling on a third-party, previously prize-winning baseline. Wildcard is the natural route to ask about, not automatic eligibility. Freeze baseline and score only the new work.

The guides require exactly one primary track. They allow declared pre-existing components, require disclosure of pre-existing product code, and describe Wildcard as an extension assessed on new work. They do not explicitly settle this third-party winner-remix scenario. [H1 §§4, 9; H2 §6]

Schedule discrepancy register

Item	Supplied guide	Public page / operating decision
Registration	22 Oct, 23:45 UTC	Page: 23:59. Register before 23:45, preferably earlier.
Judging begins	28 Oct	Page: 27 Oct, 00:00. Seek confirmation; have a demo ready on 26 Oct.
Submission	27 Oct, 23:45 UTC	Both agree. In India: 28 Oct, 05:15 IST.
Prize breakdown	Guide and overview figures	Public amounts differ. Do not base track choice on an unconfirmed split.

Build opens 22 October at 00:00 UTC / 05:30 IST and ends 27 October at 23:45 UTC: 143 hours 45 minutes. The advertised 144 hours is rounded. Planning and data prepared beforehand must be declared where applicable. [H1 §§1, 9; H3 §5; E1]

07 / SCOPE

Freeze the minimum viable product

Must ship	Definition of done
One cloud-service investigation	Six small VM histories; one target service; evidence-backed candidate ranking.
Real Zetaris integration	At least two independent physical sources joined in a real query; target three. Receipts and freshness shown.
Real Meterless integration	A sourced experiment is stored, retrieved after process restart, and visibly influences a later investigation.
Bounded agent collaboration	A critic can request new evidence or a revised candidate; at most three candidates per path (six total).
Cloud resource trial	Capacity checks plus a reproducible workload test; one scoped sandbox change and read-back.
One AI Efficiency Lab	Same-model context/redundancy changes first; locked outcome checks and held-out cases.
Cost and carbon separation	Costs carry rate provenance. Carbon remains estimated with scope/coverage, or explicitly unavailable.
Human-controlled write path	Approval bound to version and scope; stale checks, duplicate protection and recovery handling.
Evidence report	Export a reviewable HTML/PDF or Markdown report with rejected plans, assumptions and verification results.

Only after all must-ship items work

Add a second evaluated model, one authorized live GCP VM trial, or weekly report scheduling. The live VM trial can be a core target when access is ready on day one; otherwise deliver a clearly labelled local sandbox rather than imply a cloud change occurred.

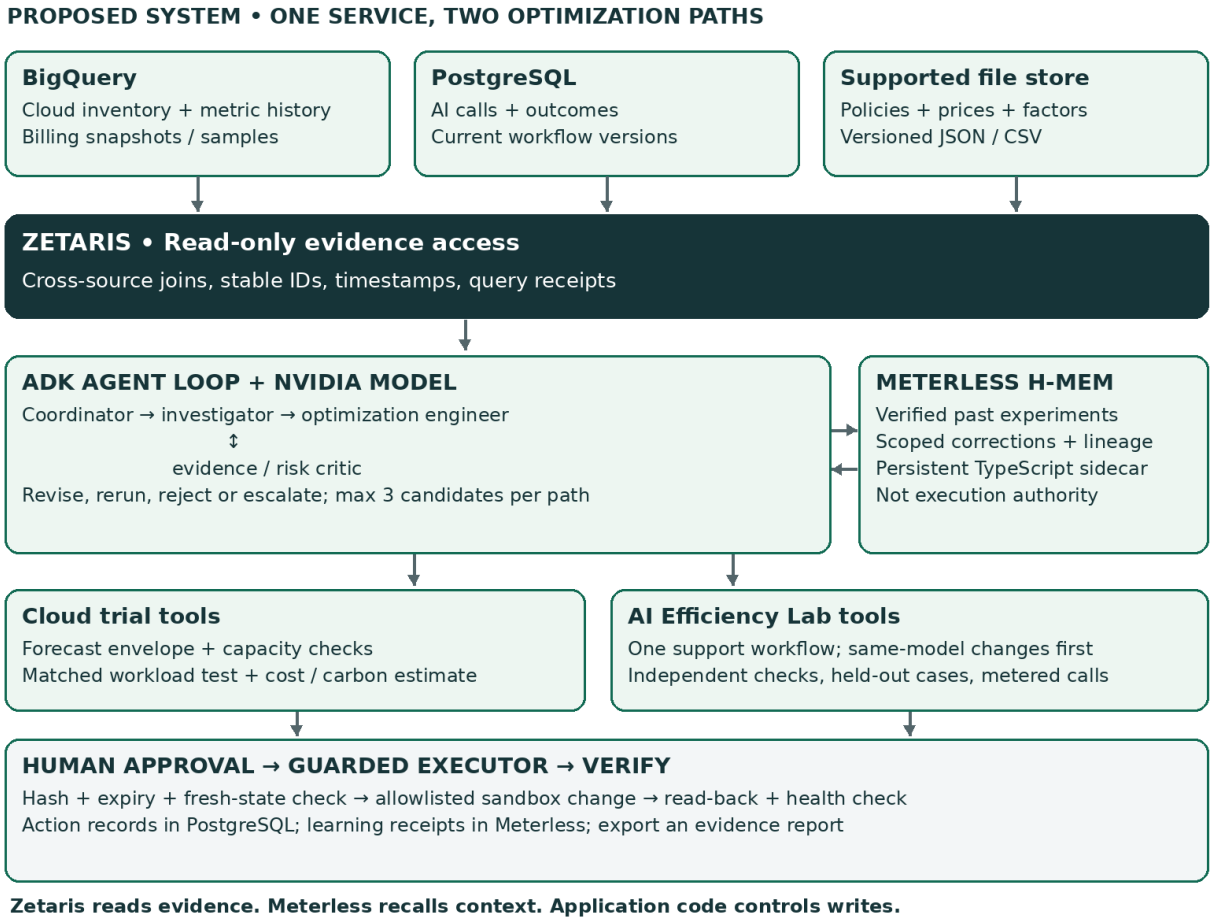
Explicitly not this hackathon

No multi-cloud migration, GPU cluster scheduler, Kubernetes fleet controller, card/subscription optimizer, autonomous production shutdown, custom model training, universal carbon accounting or elaborate slide-generation agent. Reports can be deterministic templates; they do not need another model call.

Scope protection: no new dashboard, connector or “agent” unless it strengthens the selected investigation, trial or verification. A two-builder team should keep a single model and one sandbox executor.

08 / SYSTEM DESIGN

Architecture and technology changes



Zetaris reads evidence. Meterless recalls context. Application code controls writes.

Retain Python, ADK and Streamlit. Add FastAPI for typed application endpoints. Keep BigQuery as a source where access is available, but route the cross-source investigation through Zetaris. Replace Gemini-only assumptions with a tested model adapter using NVIDIA. Add the Meterless reference as a persistent TypeScript sidecar. [R1–R3; Z1–Z3; M1–M2; A1–A2; N1–N2]

Calculators, forecasting, test runners, exports and the executor are ordinary code tools. The executor rechecks the authoritative target directly before a write; it does not rely only on a cached/federated query. PostgreSQL holds transactional run/action records. Meterless holds investigation context and learning.

Recommended first deployment: Docker Compose on one controlled host with durable volumes. Cloud Run is optional for stateless components, not a default home for a filesystem-backed memory store.

09 / AGENT BEHAVIOUR

The agents and the feedback loop

Role	Decision and tools	When it returns work
Coordinator	Scope service, horizon and constraints; select cloud or AI path; bound cost and iterations.	Escalates an unresolved gap or stops after the candidate budget.
Evidence investigator	Query Zetaris; reconcile service/resource IDs; attach timestamps; retrieve relevant Meterless history.	Fetches missing evidence rather than filling gaps with model guesses.
Optimization engineer	Propose typed candidates; invoke forecast, impact and trial tools.	Revises the candidate using a specific critic failure.
Evidence / risk critic	Check sources, constraints, independent trial output, accounting and freshness.	Returns VALID_FOR_TRIAL, REVISE, REJECT or NEEDS_REVIEW with evidence IDs.

Proposed bounded control flow

```
INVESTIGATE → PROPOSE → TRIAL → CRITIQUE
                  ↑                  |
                  └── REVISE ────────┘
PASS → APPROVAL → REVALIDATE → EXECUTE → VERIFY
FAIL / stale / missing → REJECT or HUMAN_REVIEW
```

Implement the transitions in an explicit ADK loop/custom workflow supported by the selected version. The model chooses relevant investigation tools and proposes revisions; application code limits transitions and permissions. The official ADK documentation describes loop workflows, but the installed version must be pinned and tested. [A1]

Show a decision record—not hidden reasoning

Log tool names, input hashes, source IDs, proposed configurations, failed checks, revision reasons and measured outputs. A short rationale should point to those records. Do not claim an internal model reasoning trace is proof of correctness.

Initial design limits: one service per run, three candidates per path and six total, two external-call retries, explicit per-call timeouts and a configurable experiment-spend cap. These are proposed defaults, not event rules. A timeout yields an incomplete/read-only result, never a silent approval.

10 / REQUIRED SPONSOR

Zetaris: the evidence backbone

Zetaris documents virtualized access to databases, files and REST sources, including a BigQuery connection. Use these capabilities to make the investigation depend on joined evidence—not on a ceremonial sponsor query. Availability and authentication in the provided workspace must be tested. [Z1–Z3]

Physical source	Proposed records	Important join
BigQuery / approved database	Cloud inventory, metric history and billing snapshots; synthetic samples in sample mode.	resource_id + service_id + time window
PostgreSQL	AI run/call records, outcome checks and current workflow versions.	service_id + run_id + config_version
Supported file/object store	Policies, price snapshots and carbon-method records in CSV/JSON.	policy_id + effective version; provider + region + SKU

A meaningful query

For a selected service and period: aggregate cloud cost once per resource/day; aggregate AI usage once per run; join service ownership and applicable policy versions; return independent totals and source pointers. Do not join raw time-series rows to every AI call and then sum costs: that can multiply amounts.

Connection and failure policy

Use read-only credentials, allowlisted views, row/time bounds and the workspace-supported query interface. Follow returned query pagination. Avoid unrestricted model-generated SQL. If prepared bindings are unavailable, use validated typed parameters and server-side templates, not string concatenation.

A remote Zetaris workspace cannot reach the team’s localhost by default. Put the permitted samples in sponsor-reachable storage or an approved secured database; confirm connector drivers, network rules and authentication on day one. A JDBC/file connection is acceptable if the desired REST connector is unavailable.

Sponsor proof: show the actual joined query, record counts, input versions and a result that changes when a source record changes. Offline fixtures are resilience aids, not a claim that Zetaris was running offline.

11 / REQUIRED SPONSOR

Meterless: memory that changes decisions

Use the Meterless H-MEM runnable reference through the Agentathon starter. It is a TypeScript specification/reference ecosystem, not a billing meter or ready-made production database. The starter requires a setup step that vendors the engine; persistence must survive restarts. [M1–M2]

Recommended integration

Wrap the reference in a private TypeScript HTTP sidecar with a durable volume. Expose our application-owned remember, recall and audit routes to Python. Map validated records to the actual reference services. Do not invent an official “Meterless API key” or claim our routes are vendor-hosted APIs.

Remember	How it helps later
Rejected candidate	“Target T failed the peak-load case under workload version W.” Prevents treating a failed configuration as already validated.
Approved exception	A human-confirmed workload schedule or identity correction with source, expiry and scope.
Experiment receipt	Config hash, test-set version, required checks, result and evidence locations.
Correction / supersession	A new policy or workload version invalidates the old applicability without erasing the prior record.

Applicability is an application check

Each receipt is scoped by tenant, service, resource/workflow version, policy version, model configuration and test set. Recall retrieves candidate memories; deterministic code decides whether they still apply. A higher retrieval score is not permission to execute. Current policy and current infrastructure state take precedence.

Use provenance, source references and audit history from H-MEM. Keep approval tokens, idempotency keys and transactional execution status in PostgreSQL. H-MEM explicitly separates memory from replay; do not call it the rollback engine. Limit critical memory corrections to reviewed updates. [M1]

Demo: store a failed trial, restart the service, recall it with its evidence, then change the policy version and show why a new test is required. The starter’s separate “25%” rubric must not replace this event’s 10-point Sponsor Tech criterion. [M2; H1 §12]

12 / OPTIMIZATION LOGIC

Cloud forecasting and workload checks

Do not equate low average utilization with a safe resize

For the initial dataset, use six VM histories over 21 days at 15-minute intervals: 12,096 observations before omissions. Track CPU demand, memory used, latency, errors, request count and known batch windows. The sample is small enough for deterministic replay; it is not evidence that real services behave the same way.

Start with a seasonal-naive forecast and an empirical stress envelope derived from historical variation. Backtest against withheld time windows. Retain BigQuery ML only if its models and access are already ready; Zetaris does not replace a forecasting engine. Do not train large models at demo runtime.

Candidate eligibility checks

Check	Rule to implement
Coverage and freshness	Reject missing/invalid/too-old required metrics; no default-zero CPU or memory.
Target capacity	Convert CPU fractions to approximate vCPU demand and memory to GiB, then compare with actual target capacity and a declared reserve.
Peak periods	Use known batch windows and observed peaks as well as averages. Flag unobserved future events.
Compatibility	Restrict to supported same-series non-production candidates. Exclude local SSD/GPU/architecture changes from the initial executor.
Performance	Run a matched sandbox workload and check latency, error rate and completion requirements.
Change conditions	Check allowed window, ownership, current version, trial budget and recovery plan.

Example: 10% average CPU on 8 vCPU implies about 0.8 vCPU average demand. A recorded 6.4-vCPU peak invalidates a 2-vCPU target regardless of the low average. This is a constructed test, and proportional CPU conversion is only a screening approximation—not a hardware-equivalence benchmark.

Google documents machine-type restrictions and the requirement to stop a VM for this change path. Forecasts cannot guarantee future capacity or zero downtime. A “RUNNING” status is not an application health check. [G1]

13 / ACCOUNTING

Cost calculations that can be defended

Keep three different outputs separate

Observed spend comes from a dated bill or metered run. Projected cost is the result of an explicit usage/rate scenario. Realized savings require a later comparable observation and a defensible baseline. The UI should never present a projected monthly number as money already saved.

```
cloud_cost = Σ(billable_quantity × applicable_unit_rate)
ai_run_cost = input_cost + output_cost + tool/retry_cost
net_saving = baseline_cost − candidate_cost
             − experiment_cost − transition_cost
cost_per_accepted_task = total_run_cost / accepted_tasks
```

Use fixed decimal arithmetic and explicit units/currency. Include all attempts, retries and failed tasks. If no task is accepted, cost per accepted task is undefined—not zero. Report optimization-agent usage separately so its own cost is visible.

Rates and discounts

Use a small, dated official SKU/rate snapshot or documented account rates. Google's Catalog API supplies public SKU information; account-specific prices require the appropriate account-pricing source. Public list-price savings are not necessarily cash savings under an existing commitment. [G2]

For the minimum build, exclude commitment restructuring, taxes, egress changes and shared-resource allocation unless their effects are represented. Mark excluded charges. Do not assume a stopped VM eliminates attached storage/IP costs. Do not count two mutually exclusive VM candidates as two additive savings.

Worked arithmetic — synthetic rates only

Same 100-hour horizon	Illustrative amount
Baseline VM: 100 × $0.12	$12.00
Candidate VM: 100 × $0.08	$8.00
Experiment and transition	$0.60
Scenario net saving	$3.40, assuming all usage/rates above and passed workload checks

Cloud and AI savings may be combined only when they cover the same horizon and non-overlapping bill items. Do not count a hosted model's API charge and an assumed underlying GPU bill as two independent costs.

14 / ESTIMATION BOUNDARIES

Carbon: keep it central and honest

Keep GreenOps' environmental focus, but label what can actually be known. Google's Carbon Footprint export is monthly and delayed; it cannot verify the carbon reduction from a VM change during a four-minute demo. Its reporting methodology is distinct from a prototype's scenario model. [C1–C2]

Recommended minimum methodology

Implement a narrow, versioned usage-based estimate informed by the Cloud Carbon Footprint methodology, using explicit energy and grid-factor assumptions for the selected resource/region. Report the supported scope, not a universal carbon total. Preserve the source, year, units and method version of every factor. The CCF project itself describes uncertainty in its estimates. [C3]

```
estimated_operational_kgCO2e = kWh × kgCO2e_per_kWh
scenario_reduction = baseline_estimate − candidate_estimate
# Both scenarios: identical duration, boundary and units.
```

If the method already includes facility overhead, do not add it again. If a provider returns a total for 24 hours, subtract the two 24-hour totals directly; do not multiply again by 24. Keep embodied emissions separate unless you implement and explain the allocation. A reduced allocation is not proof that manufacturing emissions were physically avoided.

Climatiq integration decision

The original code calls a legacy cloud-compute endpoint. Current account access and endpoint support were not verified in this review. Test an approved current integration on day one before treating it as a dependency. Otherwise use a clearly labelled bounded estimate or synthetic factor fixture—not fabricated API output. [R5, C4]

Cloud cost down does not automatically mean carbon down

A smaller VM may run longer; migration may add overhead; a factor may be missing. Show trade-offs or UNKNOWN. Do not convert fewer hosted-model tokens directly into kgCO2e without a defensible provider/workload method. For the AI Efficiency Lab, report calls, tokens, latency and cost; leave carbon unavailable when the necessary information is absent.

The carbon card must show: coverage, method/version, interval, units, input evidence, exclusions and "Estimated—not a verified avoided-emissions claim."

15 / AI EFFICIENCY LAB

The SpendProof feature to add

One workflow. One model to start. Independent quality checks. No general subscription manager.

Use a small support workflow that answers order-status and return-policy questions from structured order records and a versioned policy document. Run actual model calls on synthetic customer cases and capture traces. Prefer context selection and duplicate-call reduction before adding a second model.

Candidate	What is tested
Baseline	Existing workflow configuration with its actual call count, latency and accepted outcomes.
Candidate A	More aggressive context reduction. It must still retrieve every policy fact required by the task.
Candidate B	Policy-aware retrieval plus removal of duplicate, side-effect-free lookups within the same run.
Optional later	A second eligible model for a defined task class. Only after the same-model experiment and sponsor integrations work.

Test design

Create 80 labelled cases: 40 development and 40 held out. Include straightforward status questions, policy exceptions, missing information and adversarial instructions in retrieved text. The outcome verifier should check expected facts and required actions with code; use human review for ambiguous text. An LLM may assist critique but should not be the sole judge of its own optimization.

The optimizer may inspect development failures and revise at most twice. It must not inspect held-out labels while tuning. Evaluate the selected candidate on the locked held-out set, record the sample size and show failures by category. A small passing sample does not prove future equivalence.

The decisive failure path

An optimization that omits a required return-policy exception is rejected even when it uses fewer tokens. The system may find a better configuration, or conclude that no tested candidate meets the requirement. Both outcomes are valid; never script a required savings percentage.

Persist each experiment in Meterless with config, model, policy and test-set versions. A changed policy invalidates applicability, not history. Only an explicitly approved sandbox configuration can become active.

16 / CONTROLLED ACTIONS

Approval, execution and recovery

The mandatory action is a scoped, executable sandbox configuration change with read-back. Prefer a team-owned disposable GCP VM trial when access, budget and permissions are confirmed; otherwise use a real local worker configuration and label it a sandbox analogue, not a GCP resize. Do not use the upstream project or its credentials.

Stage	Required behaviour
Prepare	Save target ID, exact before/after configuration, policy/source versions, test receipts and proposed recovery.
Approve	Authenticated operator approves a specific plan hash, scope and expiry. Natural-language enthusiasm is not an execution token.
Revalidate	Read the authoritative target again. Check version, allowed project/resource, maintenance window and current policy.
Apply	Execute typed allowlisted operations only, one resource at a time. Acquire an application lock and use an idempotency key.
Verify	Check operation result, resulting configuration and application health. Compare measured performance with the approved trial conditions.
Recover	On failed verification, attempt the scoped recovery only if its preconditions hold. Otherwise stop and escalate. Do not promise successful rollback.
Record	Persist transactional status and attach verification evidence. Write a sourced learning receipt to Meterless.

GCP-specific constraints

This VM change path stops the instance, changes its machine type and starts it again. It has compatibility and availability constraints, including possible IP changes; the official instructions must be checked for the exact target. Exclude stateful production systems, local SSD, GPU and architecture changes from the first implementation. [G1]

Separate application state from memory

Use explicit statuses: DRAFT, VALIDATED, AWAITING_APPROVAL, APPROVED, STALE, EXECUTING, VERIFIED, RECOVERY_REQUIRED, FAILED. Meterless recall cannot jump the state machine or authorize a write. A missing source, stale approval or unavailable critical dependency leaves the result read-only.

A process returning success is not the final outcome. The demo must show the configuration and health read-back, or honestly leave verification pending.

17 / DEVELOPER DESIGN

Data contracts and application APIs

The following are proposed application contracts, not existing vendor endpoints. Keep payloads typed, small and versioned. Exact Zetaris/H-MEM adapter calls must follow the selected sponsor workspace/reference version.

Record	Minimum fields
EvidenceRef	id, source_system, record_key, observed_at, retrieved_at, version/hash, unit, query_receipt_id
CandidatePlan	plan_id, service_id, target_id, action_type, before, after, constraints, evidence_ids, plan_hash
TrialResult	trial_id, config_hash, workload/testset_version, measured metrics, required checks, failure_ids, total_cost
ImpactEstimate	interval, currency, baseline/candidate amounts, assumptions, rate_source, carbon_method, carbon_coverage
Approval	approval_id, actor_id, plan_hash, allowed_scope, approved_at, expires_at, status
ActionReceipt	action_id, idempotency_key, target_version, before/after, operation_id, health_readback, final_status
MemoryEnvelope	tenant/service scope, type, content, source, provenance, evidence links, applicability versions, supersession link

Minimum API surface

```
POST /investigations          create bounded investigation
GET  /investigations/{id}     findings, candidate states
POST /plans/{id}/trials       run a candidate experiment
POST /plans/{id}/approve      record scoped human approval
POST /plans/{id}/execute      revalidate and execute
GET  /actions/{id}            status + independent read-back
GET  /reports/{id}            export evidence report
```

Private sidecar routes such as POST /memory/remember, POST /memory/recall and GET /memory/audit/{id} are our wrappers over H-MEM. Bind them to the private network and authenticate callers. Production tenant isolation is beyond the minimum prototype; use one sandbox tenant and make that limit explicit.

18 / DATA PLAN

Reproducible data and demo modes

Dataset	Proposed size / purpose
Cloud history	6 VMs × 21 days × 96 quarter-hour slots = 12,096 rows before deliberate gaps.
Resource registry	Six full resource identities, allowed environments, capacities and service ownership.
Policies and factors	Small versioned catalog: maintenance windows, workload constraints, rates, estimation-method metadata.
AI evaluation cases	80 synthetic labelled cases split 40 development / 40 held out; capture genuine call traces when run.
Historical memory	Small declared seed for demonstration, plus newly generated experiment receipts clearly distinguished from seeds.

Six cloud cases to include

Eligible staging resource; low-average but high-peak worker; missing memory metrics; stale observation window; changed target after approval; unsupported target/factor. Keep input data separate from expected outcomes. The agent must compute the conclusion, not look up a scripted answer.

Two honest runtime modes

CONNECTED_DEMO uses real sponsor connections against permitted sample sources, actual model calls and an executable sandbox. REPLAY_MODE loads saved source/model observations for resilience and disables real infrastructure writes. Display the mode prominently. Replay is not a live integration demonstration.

Provide a bootstrap command that validates environment, loads samples and tests adapters. Account provisioning and secrets are external prerequisites, not something a clean-start script can magically create. Keep sponsor keys out of the repo and give judges a documented sample route.

Data and artifact hygiene

Document source/licence, generation seed, schema, version, retention and size. Use fictional customers and no real messages. Any data prepared before the window is a declared pre-existing component. The guides explicitly support lightweight samples and fallback mode; that does not waive required sponsor integration. [H3 §§5–7, 15]

Do not ship another team's screenshots or claim their historic results. Create new product visuals, new traces and new evaluation evidence during the eligible build.

19 / TEST PLAN

Evaluation: prove the new work

Compare the proposed system with two transparent baselines: (A) a simple utilization-threshold heuristic; (B) the same bounded optimizer without recalled experiment history. Do not label the heuristic as measured original GreenOps performance unless the original was actually run under a valid setup.

Test group	Acceptance scenario
T01–T03 | Data	Join source IDs correctly; prevent cost fan-out; missing metrics yield NEEDS_REVIEW.
T04–T06 | Capacity	Reject low-average/high-peak case; check target memory; block unsupported machine configurations.
T07–T09 | Accounting	Correct 24-hour units; preserve same-type/two-region results; no summing alternative savings.
T10–T12 | AI quality	Reject a missing policy exception or customer/order mismatch; keep held-out labels inaccessible to the optimizer.
T13–T15 | Memory	Recall after restart; invalidate outdated applicability; preserve correction provenance and service scoping.
T16–T18 | Execution	Block expired approval; target changes invalidate approval; repeated execution is idempotent.
T19–T21 | Reliability	An uncertain write is investigated before retry; failed health checks enter recovery; dependency outages block unsafe action.
T22–T24 | Robustness	Retrieved instructions cannot authorize a write; mismatched workload comparisons fail; NO_CHANGE is a valid result.

Metrics to publish

Report candidate decisions by scenario, false acceptances, required-check pass rate by class, accepted outcomes, total experiment spend, cost per accepted task, latency, tool/query counts, and recovery results. For carbon report coverage and methodology, not a made-up accuracy percentage.

Measure whether memory reduces repeated failed trials or tool calls under equivalent workload versions. Include the memory-read overhead. Run controls with memory disabled, keeping models, prompts and data fixed. Record limitations and sample counts; “passed our small suite” is not “production safe.”

A failed optimization can still demonstrate a correct system. Publish the rejection and its evidence instead of tuning the demo until every candidate appears successful.

20 / SPRINT PLAN

Six-day build and team ownership

Planning assumption: three builders with Python/web/API experience; a fourth builder can own evaluation and the demo. Dates below are proposed internal milestones within the published window, not additional organizer deadlines. [H1 §§5, 11]

Day	Work and owner	Exit condition
22 Oct	Data builder: Zetaris + H-MEM sidecar. Agent builder: model/tool smoke test. UI builder: minimal investigation view.	One real joined query; memory survives restart; one model tool cycle.
23 Oct	Cloud investigator, deterministic impact math and explicit constraint checker. UI shows evidence and rejected plans.	One complete cloud case, including a real rejection/revision path.
24 Oct	Workload trial runner; AI lab baseline and same-model candidates; locked checks and cost capture.	One failed AI candidate and an honestly evaluated revision or no-change outcome.
25 Oct	Approval state machine, sandbox executor, read-back, stale-state and duplicate protection.	A scoped change is verified; a stale or invalid change is blocked.
26 Oct	Freeze features. Run acceptance tests; restart/replay checks; record video; finish deck and README.	Complete submission package ready; no critical integration left mocked.
27 Oct	Fresh install using documented credentials; validate links, attribution and declarations; submit early.	Submit before 23:45 UTC; target 18:00 UTC internally.

Day-one gate and scope cuts

If BigQuery connectivity blocks progress, use another supported database/file source with real Zetaris access and document the substitution. If the model adapter blocks ADK, keep the same agent design with a thin tested provider adapter; do not change frameworks casually. If cloud write permissions are unavailable, use an explicitly labelled local sandbox and keep real sponsor integration.

Cut in this order: auto slide generation, weekly scheduling, second model, extra dashboards, live cloud resize. Do not cut the critic loop, evidence citations, required sponsor integration, independent tests, approval or verification.

21 / OPERATIONS

Deployment, access and spend controls

Keep deployment boring

Run Streamlit, FastAPI, the H-MEM sidecar and PostgreSQL with Docker Compose on one controlled host. Use durable volumes for PostgreSQL and memory. Use a single sidecar writer unless the selected reference/storage layer supports concurrency safely. Back up before memory maintenance. Do not persist critical state only on an ephemeral app filesystem.

Required pre-build confirmations

Dependency	What must be confirmed
Zetaris workspace	Endpoint, supported connectors, read-only credentials, network reachability, query limits and setup collection.
Meterless	Pinned reference/starter revision, setup/vendor process, licence notices, persistence and restart behaviour.
NVIDIA	Available model ID, endpoint, API access, tool/structured-output support and actual usage limits.
GCP / carbon provider	Own project and permissions for optional live trial; billing source; approved current carbon API if used.
Original project	Reuse permission or independent-implementation decision; organizer response on appropriate track.

Cost budget—not an assumed sponsor allowance

Set a team-approved dollar cap before running experiments. Estimate model calls × observed mean billed cost after the first smoke test, plus database/hosting and optional VM time. Include failed retries and orchestration. Sponsor credits, endpoint quotas and free access were not verified. Never present free-credit usage as proof of zero economic cost.

Proposed runtime discipline: bounded requests, rate-limit backoff, three candidates per path, six total, and cancellation between stages. Stream progress so a longer test does not appear frozen. The demo may show an earlier completed trial only if it is clearly labelled and reproducible; do not masquerade prerecorded output as a live run.

Dependency security

Choose one mutually compatible, current dependency set after a smoke test, lock it and record the versions. The ADK LiteLLM page carries a dependency security advisory; check advisories and avoid copying old or compromised pins. This blueprint does not certify a particular dependency release. [A2]

22 / PRESENTATION

A clear four-minute demo and pitch

Time	What to show / say
0:00–0:25	Problem: “A cheap change can break a service or an AI answer. We investigate and test before it becomes active.” Show the selected service.
0:25–0:55	Real Zetaris query across sources. Open one evidence receipt and the candidate queue.
0:55–1:35	Reject the low-average/high-peak resize; show specific constraint evidence. Revise to a separate staging candidate.
1:35–2:10	AI lab: show the failed policy-exception candidate and the measured result of the revised configuration.
2:10–2:55	Human approval; fresh-state check; scoped sandbox action; resulting configuration and health verification.
2:55–3:30	Restart/reopen. Meterless recalls the experiment with its source; changed policy makes old validation inapplicable.
3:30–3:55	Results and limitations: measured vs projected, carbon scope, and what is new versus the original GreenOps inspiration.

One-minute pitch draft

“GreenOps Verify helps an engineering lead remove cloud and AI waste without confusing a cheaper configuration with a valid one. We keep the GreenOps idea of audit, forecast, optimization and reporting, and add a test-and-verification loop. Zetaris connects the source evidence. Meterless preserves what earlier experiments established and why. Our agents challenge proposals, revise failed candidates, and ask for approval before a scoped change. The outcome is not just a recommendation: it is an inspectable change record with the test result, financial assumptions, estimated carbon coverage and verified final state.”

Suggested 10-slide deck

1 Problem/user. 2 Original inspiration and new contribution. 3 Product workflow. 4 Agent feedback loop. 5 Zetaris integration. 6 Meterless memory. 7 Cloud trial and AI lab. 8 Actual evaluation results. 9 Approval, limitations and recovery. 10 Roadmap and team. Replace all future-tense result claims with measured evidence before submission.

23 / EVENT FIT

Submission and originality checklist

The guide requires a 1–4 minute demo, slide deck, repository with setup/usage/dependencies, and an explanation of Zetaris, Meterless, Cursor and NVIDIA use. The product must be functional and pre-existing components must be declared. [H1 §§2, 8–9]

Rubric	Evidence to prepare
Impact — 30	One understandable user and costly decision; operator feedback if available; no invented ROI.
Technical — 20	Cross-source queries, deterministic checks, executable action, restart and failure handling.
Innovation — 15	New proof-before-change loop, quality-tested AI lever and version-aware learning—not merely original GreenOps features.
Demo — 15	A rejection, a revised plan, a verified action and a visible sponsor-dependent result.
Product / UX — 10	Understandable candidate cards, evidence, assumptions, clear status and scoped approval.
Sponsor Tech — 10	Real Zetaris join, real H-MEM remember/recall/audit, Cursor development record and tested NVIDIA contribution.

Attribution and build provenance

Create PREEXISTING.md, ATTRIBUTION.md and NEW_WORK.md. Identify conceptual inspiration separately from copied components. If permission permits reuse, record exact paths, upstream commit and permission/licence terms; never describe the original authors' award as ours. A code-diff or tagged baseline should show what was created during the eligible window. [H1 §9; H3 §19]

Use Cursor as the development environment and document the actual workflow; a screenshot without substantive development is not the goal. Use NVIDIA in the running investigation or experiment, not only in a logo. Retain source licences and keep secrets out of prompts, screen recordings, logs and example environment files.

Do not submit this planning pack as evidence that integrations or savings already work. The final submission must add actual implementation, traces, tests, measured results and working access instructions.

24 / DECISION REGISTER

Risks, fallbacks and claims

Risk	Response / honest boundary
No permission to reuse	Independent implementation; no copying code/assets. Keep inspiration attribution and ask organizers about track.
Cloud access unavailable	Real local sandbox plus cloud scenario analysis. Do not label it a live VM resize.
Zetaris unreachable	Expose degraded mode and disable writes. Retain previous genuine receipts; resolve the integration rather than claim fixture access is sponsor access.
Memory store lost/unavailable	Restore a documented backup where possible; fail read-only if required context cannot be checked. Do not regenerate “past approvals” with a model.
Carbon factors insufficient	Show UNKNOWN/partial coverage. No token-to-carbon shortcut, no certified-avoidance claim.
No cheaper quality-passing AI config	Return no-change with evidence. That is a valid result, not a reason to lower checks after seeing results.
No reliable peak history	Escalate the cloud proposal; missing peak evidence is not evidence of spare capacity.
Recovery cannot restore state	Stop; preserve evidence; require human handling. Never roll back unrelated legitimate changes.
Small/biased test set	Disclose coverage and holdout limits. Avoid production-safety or guaranteed-quality language.

What makes this version defensible

The proposed contribution is a single, tested decision process across cloud and AI changes: inspectable data, explicit constraints, rejected experiments, scoped memory and verified actions. No claim is made that this combination is globally novel or certain to win. The judging case depends on actual execution.

Questions for onboarding

Ask about third-party winner-code reuse and Wildcard eligibility; which deadline/judging schedule governs; sponsor connector availability and credentials; whether an executable local sandbox is acceptable; and how to disclose pre-event plans/data. Do not let a rules uncertainty turn into an unreported assumption.

25 / DEVELOPER START

File-level implementation map

This is a conceptual migration map. Use original files only when permission permits; otherwise create the proposed modules independently. The companion Developer Handoff expands contracts, tests, work items and a ready-to-use Cursor brief.

Original module / concept	Proposed build
greenops_agent/agent.py	agents/coordinator.py + workflow.py: bounded stateful investigation, current model adapter, lazy client startup.
optimization_advisor_agent	agents/investigator.py + optimizer.py + critic.py: replace one-way analysis with revision based on tests.
forecaster_agent/agent.py	tools/forecast.py: typed histories, seasonal baseline and empirical stress envelope; no hardcoded model/project path.
impact_calculator_agent/agent.py	tools/cost.py + carbon.py: deterministic decimal math, unit-safe interval handling and full scenario keys.
safe_executor_agent/tools.py	services/approval.py + executors/: allowlisted writes, idempotency, fresh-state checks, verify/recover.
Reports / frontend	Streamlit investigation desk and deterministic report templates; original screenshot/template assets not copied.
New sponsor modules	adapters/zetaris.py and services/hmem-sidecar/: real source queries, persistent memory and audit receipts.
New AI lab	experiments/support_workflow.py + verifier.py: cases, traces, independent checks and configuration comparisons.

First five work items

1 Resolve baseline/track rights. 2 Run sponsor/model connection smoke tests. 3 Freeze typed evidence and plan contracts. 4 Build one cloud investigation through a rejected candidate and revision. 5 Create the approval/verification scaffold before adding UI polish.

One repository, one minimal deployment, one cloud target service, one AI workflow. Protect the connected evidence → test → approved change → verified outcome path.

26 / REFERENCES

Sources and review boundaries

Primary materials reviewed on 16 October 2026. Repository findings are pinned to the stated GreenOps commit; sponsor documentation may change. The six supplied guides include duplicate solutioning material; duplicate files do not count as independent sources.

[H1] Open Agent Hackathon 2026 — Welcome Guide

User-provided guide; especially §§2–5, 8–9 and 12. Sponsor requirements, dates, deliverables, pre-existing work and judging. No public URL supplied.

[H2] Open Agent Hackathon 2026 — Hackathon Overview

User-provided guide; especially §§4, 6, 11–14. Agent behaviour, track selection, development and evidence requirements.

[H3] How to APPROACH Solutioning For Your Track

User-provided guide; especially §§5–9, 15 and 19. Sample-mode guidance, focused scope and declaration of pre-event data.

[H4] Open Agent Hackathon 2026 — Event Calendar

User-provided guide; deadlines and HackOS announcements as the source of updates.

[E1] Open Agent Hackathon 2026 — public event page

Live page inspected on 16 October 2026. Some dates and prize amounts differ from the supplied guides; see the discrepancy register.

[P1] GreenOps — original Devpost submission

Original creators' description, architecture and Asia-Pacific regional recognition. Descriptive claims are not independent test results.

[R1] GreenOps — repository README and baseline

Baseline branch master at commit db38811fb653631892e6cbbe1275793bd9906858. Static review, not an executed deployment.

[R2] GreenOps — coordinator

Root agent routing and prompt-level confirmation; startup secret loading.

[R3] GreenOps — optimization advisor

SequentialAgent definition for the scout/profiler/recommender workflow.

[R4] GreenOps — executor tools and agent

Forecast-average gate, VM stop/resize/start, hardcoded project and unguarded low-level change tool.

[R5] GreenOps — impact calculator

Scraped pricing, duration handling and machine-type-only result keys.

[R6] GreenOps — forecasting module

Direct BigQuery queries, specific model paths and pivoted date output.

[L1] GitHub — licensing a repository

Explains public visibility versus permission to modify and redistribute. Resolve reuse rights before including third-party code.

[Z1] Zetaris — connecting to your data

Documents virtualization-first access and database, file, streaming and REST source categories.

27 / REFERENCES

Technical reference directory

[Z2] Zetaris — Google BigQuery connector

Documents BigQuery connectivity, JDBC setup and authentication requirements.

[Z3] Zetaris — REST access

API access, bearer-token handling and query pagination. Use the collection for the supplied workspace version.

[M1] Meterless H-MEM — engine README

Specification plus runnable TypeScript reference; persistence, provenance, retrieval and trust ledger. Not a production SDK or a replay engine.

[M2] Meterless — Agentathon starter

Setup vendors the reference engine; demonstrates cross-run persistence. Its own rubric is not this event's rubric.

[A1] Google ADK — loop workflows

Bounded iterative agent workflows. Confirm the supported interface in the dependency version chosen for the build.

[A2] Google ADK — LiteLLM connector

External-model integration and a current dependency security advisory; do not reuse old pins blindly.

[N1] NVIDIA — NIM LLM APIs

Hosted chat-completion interface. Confirm model access, structured responses and tool behaviour with the team's credentials.

[N2] LiteLLM — NVIDIA NIM provider

Provider routing and environment configuration; verify the selected ADK/LiteLLM/model combination.

[G1] Google Cloud — change a VM machine type

Stopped-instance requirement, compatibility limitations, availability risks and operational considerations.

[G2] Google Cloud — Billing Catalog API

Public SKU prices and the distinction from account-specific contract prices.

[C1] Google Cloud — Carbon Footprint exports

Monthly reporting and export timing; not an immediate per-change measurement.

[C2] Google Cloud — carbon reporting methodology

Provider methodology and accounting boundaries; do not equate a prototype estimate with provider-reported emissions.

[C3] Cloud Carbon Footprint — methodology

Usage-based operational and embodied estimation; openly states uncertainty and limitations. Use a bounded, versioned methodology.

[C4] Climatiq — API deprecation calendar

Use current account-supported APIs. The legacy cloud-compute endpoint in GreenOps was not verified as currently available.

Unverified items: provisioning and quotas for this team; private HackOS announcements; third-party-code eligibility; current support for the legacy Climatiq endpoint; actual model compatibility in the proposed stack; savings, accuracy and production readiness. All are explicit implementation or organizer checks, not assumed facts.